Blind Auction Figgie

Pricing information in a market game: the playable game, how it works, and how I tested it

Prachet Poddar | UCLA, B.S. Physics and B.S. Mathematics (expected June 2028) | October 2026

1. Summary

Figgie is a card game about trading when the deck is uncertain. All of its information is free: you see your own hand and learn the rest from other players' trades. Real traders often pay for information before they know what it will say, and the act of paying can give away their position. Blind Auction Figgie adds that decision. Before each trading window, every player can buy information at a sealed-bid data desk, and everyone sees who bought what and at what price, but not what the buyer saw.

I built it as a playable web game against three bots, with a live market, saved games, a rating and a leaderboard. I then tested the rules with headless simulations that run the game's own bot code, about **176,000 rounds** in a seeded, reproducible series. The design changed three times because of what the tests showed. An independent review of the simulator found one bug that changed the final balance. I fixed it and re-ran the series, and every number below comes from the fixed code unless marked otherwise.

Result. In the final rules, buying information pays, but nobody can buy their way to a large edge. Across about 8,000 rounds per style in mixed tables, the heaviest buyer (Info-hungry) finishes 4.2 ± 2.3 chips a round ahead of the value-based bot (Equilibrium) at the same tables, and Equilibrium finishes 3.9 ± 2.1 ahead of a bot that never buys. Three styles each finish top in 27 to 35% of their rounds.

2. The game

- Standard Figgie deck and payout, four players. 40 cards: one suit has 12, one has 8 and two have 10. The goal suit is the other suit of the same colour as the 12-card suit. Everyone antes 50 of 350 chips, so the pot is 200. At the end each goal-suit card pays 10 and the player holding the most goal-suit cards takes the rest of the pot, split on ties.
- Four trading windows of 60 seconds. Players post bids and offers, or trade at the best price. One card changes hands per trade, and any trade cancels all of your resting quotes (Figgie's rule).
- **The data desk** opens before each window and sells four widgets, each in its own sealed auction with a house reserve:

Widget	What the buyer sees	Lots	Reserve
Peek	One random card from a random opponent, never one already shown to you	2	0.5
Scan	Five random cards from the opponent you pick	1	1.5
X-ray	The opponent's whole hand, kept current through the public trade tape	2	3
Tracker	A live grid of every player's net buys and sells (an aid for people; bots already track the tape)	1	5

Table 1. The desk. Each lot goes to the highest sealed bids; every winner pays the clearing price, which is the best losing bid or the reserve, whichever is higher.

- Payments go to the house, so the players as a group always lose exactly what they spend. Information moves chips between players; it creates none. A purchase can only beat its cost at the other players' expense.
- After each desk everyone sees the buyers, whose hand each Scan or X-ray targeted, and the prices. Only the buyer sees the cards. A player may then say what it saw, truthfully or not.

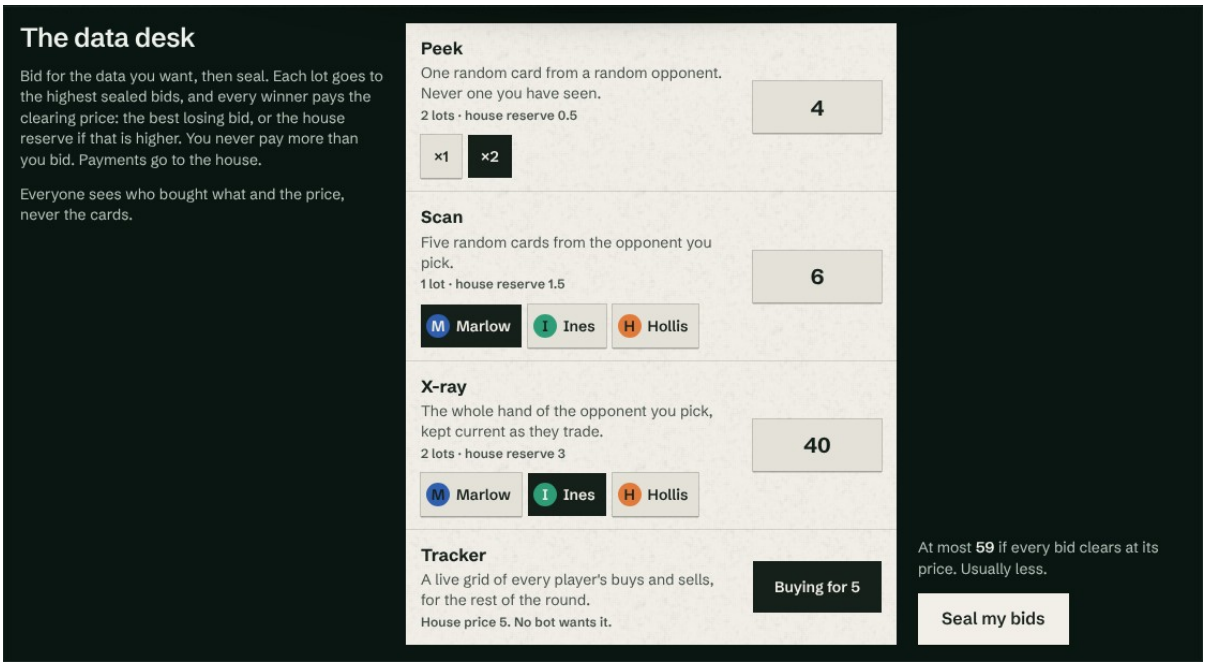


Figure 1. The data desk in the playable game, with bids entered for two peeks, a Scan of Marlow, an X-ray of Ines and the tracker.

3. The mechanism

What each player knows

There are 12 possible deck layouts: 4 choices for the 12-card suit times 3 for the 8-card suit. Each bot keeps a probability for every layout. It starts from the likelihood of its own hand, which is multivariate hypergeometric in the layout's suit counts, and updates by Bayes' rule on everything it observes:

- **Desk reveals.** Seeing suit counts v among the cards it cannot see multiplies each layout's weight by the hypergeometric probability of v given the unseen counts. An X-rayed hand is then kept exact through the public tape.
- **Trades.** When a player starts a trade by buying suit s, layouts where s is the goal gain weight; selling lowers it. The factors were measured in an earlier research simulator: about 1.7 for a buy and 1.17 for a sale by a player who just bought data, and 0.59 and 0.81 (that is, the opposite direction) for a player who did not. Reading trades the wrong way round was one of the early bugs that controls caught (section 5).
- **Claims.** A claim that a peeked card was suit s is mixed in with a trust weight t, from 0 (ignored) to 0.5 (Naive).

The value of information

I measure information in accuracy points: one point is one percentage point added to the chance of naming the goal suit correctly. From a player holding 10 cards, each additional unseen card adds close to one point, so the old yes/no answer about a suit carried about as much as 18 more cards:

Information	1 card	3 cards	5 cards	10 cards (X-ray)	20 cards	Old answer: "Is X the 12?"
Accuracy points	1.0	2.8	4.8	10.0	25.4	22.0

Table 2. Expected gain in the probability of naming the goal suit, ×100, from the opening position, 20,000 sampled deals.

A bot converts points to chips at a rate of 0.75, 0.64, 0.53 and 0.49 chips per point in windows 1 to 4, measured earlier by replaying identical deals with and without information delivered. Its bid for a widget is that chip value times a style multiplier. One and two cards are valued exactly; Scans and X-rays by sampling 48 outcomes.

How bots trade

A bot's value for one more card of suit s is 10 times the probability that s is the goal, plus the expected change in its share of the majority bonus. The bonus term comes from 40 sampled layouts and deals of the unseen cards to

opponents. It quotes 6 chips either side of that value, takes any quote that beats its value by more than 1 chip, and quotes recent data buyers the value it would have after their trade, as a market maker widens against informed flow. In the final rules, bots that buy data also blend in a value that holds X-rayed hands fixed, at weight 0.2 or 0.3. Every bot acts at the same rate: one random bot acts each second.

Style	Buys data	Reads trades	Talks	X-ray use in values
Equilibrium	At its value	Yes, and protects	Says nothing	0.2
Info-hungry	At 1.5 times its value	Yes, and protects	Says nothing	0.3
Bluffer	At its value	Yes, and protects	Lies about peeks; ignores claims	0.2
Cautious	Never	Yes, and protects	Says nothing	None
Naive	At its value	No; quotes tight	Tells the truth; believes claims	0.2

Table 3. The five bot styles. Each is one parameter change away from a shared core.

4. Tests

All runs use four bot seats. A *mixed* round draws four of the five styles at random, so every style meets the other four equally often. Two further fields put one focal bot (Equilibrium or Info-hungry) against weak opponents (Cautious, Naive, Bluffer) or strong ones (Cautious, Bluffer, Equilibrium). Seeds are fixed, so every run reproduces exactly. Intervals are 95%. Differences between two styles are measured at the same tables, as the mean of the paired difference.

Stage 1: research simulator

Before writing the game, I built a separate Python simulator with eight bot styles to test the rules on paper. Its main findings, which the game kept, were: a bot that trades faster after winning information wins only because the tick market gives it extra turns; bids reveal hands, so coarse bidding is the right response when bids are public; and letting players watch an auction for free rewards never bidding, which a pay-to-watch rule fixed. That rule set, a sealed auction for the answer to "Is suit X the 12-card suit?", became the first version of the web game.

Stage 2: the answer auction in live play

In the web game the answer auction let one style dominate. With the chip rates the game shipped with, Info-hungry led every mixed table at +8.5 ± 1.2 chips a round and finished top in 40% of its rounds. The lead depends on the chip rate. The research simulator had found that the right bid is about 1.33 times the plain value of an answer, because a rival who wins it also costs you; at the game's rates, Info-hungry's 1.5× sits near that price. Under an earlier calibration with rates about a third higher, every honest bid rises and the lead disappears (−3.8 against Equilibrium's −4.4).

Playing it showed a second problem that calibration cannot fix. One answer carries 22 accuracy points, as much as seeing about 18 extra cards. A single "no" rules out a goal suit and a "yes" settles the round, so after one or two answers the game reduces to trading two suits. I replaced the answer with the data desk, which sells evidence in pieces of 1, 5 and 10 cards.

Stage 3: the data desk

Version	Change	Info-hungry	Equilibrium	Bluffer	Cautious	Naive
v3 answer auction	As shipped	+8.5	−0.5	−5.0	−11.9	−61.4
V16 desk*	First desk; bots bid half value	−1.5	+1.2	+1.1	+1.8	−20.5
V17 desk*	Info-hungry uses X-rays	−1.5	+0.8	−0.4	+0.9	−17.8
V18 desk*	Cheaper X-rays; all buyers use them	+2.1	+1.7	+1.0	−2.0	−30.5
V19 final	Peek bug fixed; bots bid full value	+1.5	−2.0	−3.1	−5.1	−43.9

*Table 4. Net chips per round by style, mixed tables, 4,000 to 8,000 rounds per cell (intervals ±0.9 to 1.6). * Run before the bug in section 5 was fixed.*

V16 removed Info-hungry's lead but made information nearly useless: Cautious, which never buys, tied the buyers. Bots rarely met the X-ray reserve, and what they bought hardly changed their prices. V17 gave Info-hungry alone a way to use X-rayed hands; the change looked like a gain in a first, unseeded run and did not replicate in the seeded series. V18 lowered the X-ray price, added a second lot and let every buyer use X-rayed hands, which moved Cautious behind the buyers in both my own play and the simulation.

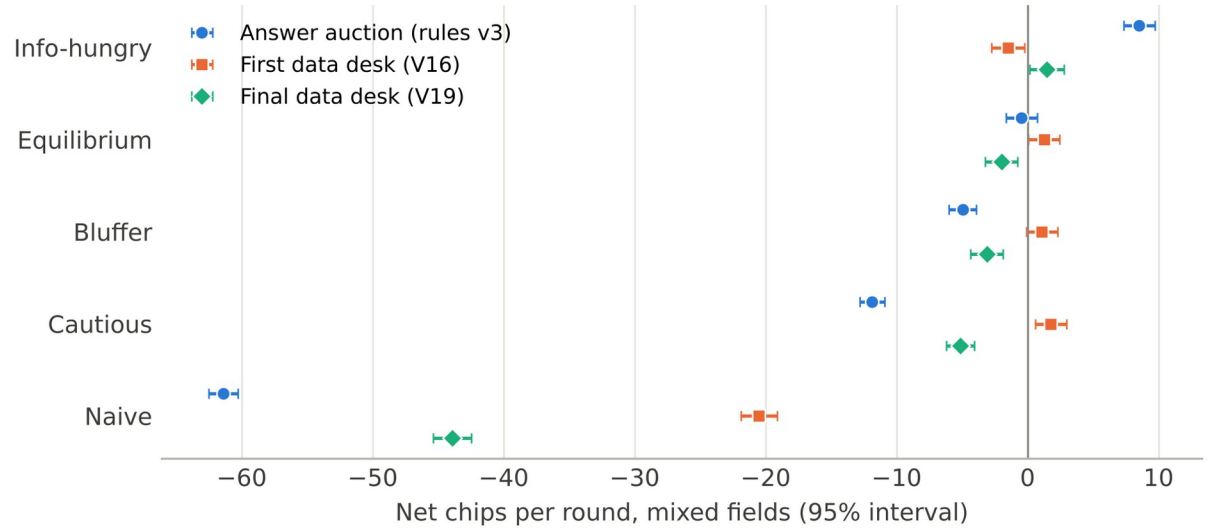


Figure 2. Net chips per round with 95% intervals: the answer auction, the first desk and the final desk.

Final rules (V19)

Style	Net per round	Rounds finished top	Paid for data
Info-hungry	+1.5 ± 1.3	35%	21.1
Equilibrium	−2.0 ± 1.2	28%	9.8
Bluffer	−3.1 ± 1.2	27%	9.9
Cautious	−5.1 ± 1.1	19%	0.0
Naive	−43.9 ± 1.5	16%	11.6

Table 5. Final rules, mixed tables, about 8,000 rounds per style. With five styles at four seats, equal styles would each finish top in about 25% of their rounds.

Comparison at the same tables	Paired difference
Info-hungry minus Equilibrium	+4.2 ± 2.3
Equilibrium minus Cautious (does buying data pay?)	+3.9 ± 2.1
Bluffer minus Cautious	+3.2 ± 2.1
Bluffer minus Equilibrium (lying about peeks)	−1.6 ± 2.3
Cautious minus Naive (reading trades and quoting wide)	+41.4 ± 2.4

Table 6. Chips per round, 95% intervals, over the 6,000 or so rounds in which both styles sat.

Against weak opponents Equilibrium and Info-hungry tie (+4.6 and +4.2). Against strong readers Info-hungry does better (−4.7 against −9.2), probably because buying more data is how a bot keeps up with players who read the tape well. Naive loses heavily in every field: it believes every claim, announces what it saw, and quotes tight against informed traders.

Design checks on the final rules

Setting	Info-hungry	Equilibrium	Cautious	Equilibrium minus Cautious
Bots bid half the value of data	+9.4	+1.0	−5.8	+8.5 ± 2.7
Bots bid full value (final)	+1.5	−2.0	−5.1	+3.9 ± 2.1
Bots bid 1.25 times value	−3.8	−4.5	−5.5	+2.2 ± 2.8
No bot uses X-rayed hands in its values	+0.1	−4.8	−4.4	+1.3 ± 2.6
Every bot relies fully on X-rayed hands	−25.7	−0.3	+20.2	−19.7 ± 5.8

Table 7. Mixed tables, 3,000 to 8,000 rounds per style in each row. Only the changed setting differs from the final rules.

Three points follow. First, at half value data is underpriced and the heaviest buyer takes a large edge; at 1.25 times value the desk prices away the gain from buying and Cautious ties the buyers again, so full value is the setting where buying is a real decision. Second, information only pays when it changes how a bot trades: if no bot uses X-rayed hands, Equilibrium is no better than Cautious. Third, full reliance on a known hand loses badly, most of all for the heaviest X-ray buyer. Holding X-rayed hands fixed treats the race for the majority bonus as static, but the other players keep trading for the same cards. A partial weight is the better estimate.

5. What the controls caught

Several results did not survive a control. Each one changed the design, so I list them here:

Apparent finding	Control	What it really was
Overbidding for answers wins	Speed boost given with and without an answer	Extra turns in my market model, not information
Overbidding still pays	Measured how much each trade reveals	Bots were reading uninformed trades backwards
Info-hungry dominates the web game	Reran at a second chip-rate schedule	Its 1.5× overbid sat near the right price at the game's rates
V17 makes Info-hungry stronger	Seeded rerun at full size	Noise; it did not replicate
V18 is balanced	Independent review of the simulator	A bug: 26% of peeks drew from hands already X-rayed
Bots should value known hands exactly	Full reliance against a partial weight	The static estimate overstates the majority race

Table 8. Each row was settled by a paired test on identical deals, or by a seeded rerun.

The last bug matters most for this document. A Peek could land on a hand the buyer had already X-rayed, so the buyer paid for a card it already knew and its beliefs counted that card twice. With the bug fixed, data became far more valuable, Info-hungry reached +9.4 a round at the old half-value bids, and the balance I had accepted turned out to depend on the bug. The fix took one line, a regression test that fails on the old code, and a recalibration from half-value to full-value bids.

6. Limits

- The bots are assumptions about how people play, not measurements. The results show how these five styles fare against each other, in a field where Naive sits at three of every four tables.
- The market is a simple tick model: one random bot acts each second, and the live game has one human seat. Results that depend on fine trading details are the least certain.
- Bots value Scans and X-rays from 48 samples, so their bids carry noise of about 1.3 and 1.8 points. Auction winners therefore pay slightly more than the data's value to them.
- Human play so far is my own. The next step is a playtest with other people, for which the game already saves every round, bid and trade.

7. Reproducing the results

`python3 run_series.py` runs rules v3 to V18, and `SERIES=v19 OUT=results_v19.json python3 run_series.py` runs the final rules, about 25 minutes each on two cores. The simulators load the game's own bot code. `node tools/test_reveal.js` is the regression test for the peek bug, and six browser test suites check the game at phone and desktop widths. The playable game, code and logs are available on request.